

link data:

<https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset?select=test.csv>

sử dụng file .ipynb trên máy local làm

Bước 0 Giữ nguyên hai file gốc

Tạo cấu trúc:

```
data/
├── raw/
│   ├── train.csv
│   └── test.csv
├── processed/
├── manifests/
└── reports/
```

Đặt hai file ban đầu vào:

data/raw/train.csv

data/raw/test.csv

Tuyệt đối không chỉnh sửa trực tiếp hai file này.

Tạo checksum:

sha256sum data/raw/train.csv

sha256sum data/raw/test.csv

Lưu kết quả vào:

data/manifests/raw_checksums.txt

Điều kiện hoàn thành

- Hai file gốc còn nguyên.
- Có SHA-256 cho từng file.
- Mọi dữ liệu mới đều được ghi vào data/processed/.

Bước 1 Đọc và kiểm tra cấu trúc CSV

Mỗi file phải có đúng ba cột:

Class Index
Title
Description

Với mỗi dòng, kiểm tra:

Có đúng ba trường hay không
Class Index có đọc được thành số nguyên không
Title có bị thiếu không
Description có bị thiếu không
Có dòng CSV bị vỡ do dấu phẩy hoặc dấu ngoặc kép không

Tạo một cột định danh riêng:

train_000000
train_000001
...

test_000000
test_000001
...

Tên cột:

row_id

Không dùng số thứ tự của DataFrame sau khi lọc làm ID, vì ID sẽ thay đổi khi xóa dòng

File đầu ra

data/reports/schema_report.json

Nội dung tối thiểu:

```
{  
  "train_rows": 120000,  
  "test_rows": 7600,  
  "train_columns": [  
    "Class Index",  
    "Title",  
    "Description"  
  ],  
  "malformed_rows": 0,  
  "missing_titles": 0,
```

```
"missing_descriptions": 0  
}
```

Điều kiện PASS

Malformed rows = 0

Missing title = 0

Missing description = 0

Nếu bất kỳ giá trị nào khác 0, dừng lại để kiểm tra trước khi tiếp tục.

Bước 2 Chuyển nhãn từ 1–4 thành 0–3

Nhãn gốc trong hai file là:

1 = World

2 = Sports

3 = Business

4 = Sci/Tech

Trong PyTorch, CrossEntropyLoss với bốn class yêu cầu target nằm trong:

0, 1, 2, 3

Do đó phải chuyển:

label = original_class_index - 1

Mapping sau chuyển đổi:

0 = World

1 = Sports

2 = Business

3 = Sci/Tech

Nên giữ hai cột

original_class_index

label

Ví dụ:

original_class_index	label	label_name
1	0	World
2	1	Sports
3	2	Business
4	3	Sci/Tech

Kiểm tra bắt buộc

```
assert set(labels).issubset({0, 1, 2, 3})
```

Đồng thời kiểm tra số mẫu từng class.

Điều kiện PASS

- Không có label ngoài 0–3.
- Không có label bị thiếu.
- Tổng số mẫu sau remap chưa thay đổi.
- Mỗi class vẫn có số lượng hợp lý.

Bước 3 Khóa một hàm làm sạch văn bản duy nhất

Đây là bước rất quan trọng. Tất cả kiểm tra duplicate và overlap sau này phải dùng cùng một hàm làm sạch.

Tạo:

```
def clean_text(text: str) -> str:  
    ...
```

Thứ tự xử lý

3.1. Chuyển text thành string và strip

```
text = str(text).strip()
```

3.2. Chuẩn hóa Unicode

Dùng:

```
unicodedata.normalize("NFKC", text)
```

Mục đích là chuẩn hóa các ký tự có hình thức khác nhau nhưng ý nghĩa tương đương.

3.3. Sửa numeric HTML entity thiếu dấu &

Trong dataset có dạng:

```
#39;  
#36;  
#151;
```

Phải sửa thành:

```
&#39;  
&#36;  
&#151;
```

Có thể dùng:

```
re.sub(r"(?<!&)#(\d+);", r"&#\1;", text)
```

3.4. Giải mã HTML entity

Sau khi sửa, dùng:

```
html.unescape(text)
```

Ví dụ:

```
doesn #39;t → doesn't  
#36;10 → $10  
#151; → —  
&amp; → &
```

3.5. Xử lý HTML tags

Ví dụ:

```
<strong>Opinion</strong>
```

phải trở thành:

Opinion

Giữ visible text, chỉ bỏ markup.

Với ticker như:

<MSFT.O>

<AAPL.O>

nên chuyển thành:

MSFT.O

AAPL.O

thay vì xóa hoàn toàn.

3.6. Xử lý backslash artifact

Dataset có các dạng:

dwindling\band
worries\about
company,\which

Các dấu \ này phải được thay bằng khoảng trắng.

Trước đó nên xử lý một số dạng escape có ý nghĩa:

\\$ → \$

\ " → "

\ ' → '

Sau đó thay các backslash còn lại:

```
text = text.replace("\\", " ")
```

3.7. Chuẩn hóa khoảng trắng

```
text = " ".join(text.split())
```

Không được xóa

- Dấu câu.
- Số.
- Phần trăm.
- Ký hiệu tiền tệ.
- Tên công ty.
- Tên quốc gia.
- Tên người.
- Từ viết tắt.
- Reuters, AP hoặc tên nguồn tin trong thí nghiệm chính.

Lưu cả raw và clean

Các cột nên có:

title_raw
description_raw
title_clean
description_clean

Điều kiện PASS

Sau cleaning:

- Không còn numeric entity dạng #39;.
- Không còn backslash nằm giữa từ.
- Không còn HTML tag thực.
- Không có text rỗng.
- Không làm mất số hoặc tên riêng.
- Một số mẫu được kiểm tra thủ công trước và sau cleaning.

Bước 4 Kiểm tra cleaning bằng mẫu thủ công

Không được tin hoàn toàn vào code cleaning chỉ vì nó chạy không lỗi.

Lấy ngẫu nhiên:

50 mẫu World
50 mẫu Sports
50 mẫu Business
50 mẫu Sci/Tech

In song song:

Title raw

Title clean
Description raw
Description clean
Label

Đặc biệt kiểm tra những mẫu chứa:

\
#39;
#36;
&

<MSFT.O>
\$
%
số thập phân

Tiêu chí đánh giá

Cleaning tốt khi:

- Câu dễ đọc hơn.
- Không làm thay đổi ý nghĩa.
- Không nối hai từ lại với nhau.
- Không xóa entity quan trọng.
- Không biến ký hiệu tiền tệ thành khoảng trắng.
- Không tạo text rỗng.

File đầu ra

data/reports/cleaning_samples.csv
data/reports/cleaning_report.json

Bước 5 Tạo canonical key và hash

Duplicate không được kiểm tra trực tiếp trên raw text vì cùng một bài có thể khác nhau chỉ do khoảng trắng hoặc HTML entity.

Tạo bản canonical chỉ dùng để so sánh:

```
canonical_title = clean_title.casefold()  
canonical_description = clean_description.casefold()
```


Tạo khóa:

```
canonical_text = (  
    canonical_title  
    + "\n"  
    + canonical_description  
)
```

Sau đó tạo SHA-256:

```
text_hash = sha256(  
    canonical_text.encode("utf-8")  
)hexdigest()
```

Lưu ý quan trọng

casefold() chỉ dùng để tạo khóa duplicate.

Text đưa vào BERT vẫn dùng title_clean và description_clean, không phải canonical lowercase tự tạo.

Không được làm

Không xác định duplicate chỉ bằng Title, vì nhiều tin khác nhau có thể dùng cùng một tiêu đề.

Exact duplicate chỉ được xác định khi:

Title clean giống nhau

VÀ

Description clean giống nhau

Bước 6 Audit exact duplicate trong từng file

Nhóm các dòng theo:

text_hash

Chia thành hai loại.

6.1. Duplicate cùng nhãn

Ví dụ:

Cùng title
Cùng description
Cùng label

Trong bộ dữ liệu clean dùng để phát triển model:

- Giữ dòng có row_id nhỏ nhất.
- Loại các bản sao còn lại.
- Ghi đầy đủ vào báo cáo.

6.2. Duplicate khác nhãn

Ví dụ:

Cùng title
Cùng description
Một dòng Business
Một dòng Sci/Tech

Đây là label conflict.

Quyết định an toàn:

Loại toàn bộ nhóm conflict khỏi tập dùng để train và validation.

Không chọn ngẫu nhiên một nhãn. Không lấy nhãn xuất hiện trước. Không tự sửa nhãn bằng cảm tính.

File báo cáo

data/reports/exact_duplicates.csv
data/reports/conflicting_duplicates.csv
data/manifests/removed_rows.csv

removed_rows.csv cần có:

row_id
text_hash
original_label
reason
representative_row_id

Reason:

same_label_duplicate
conflicting_label_duplicate

Lưu ý

Số duplicate chính xác phụ thuộc vào hàm cleaning. Vì vậy không nên xóa dữ liệu dựa trên các con số kiểm tra sơ bộ trước đây. Hãy khóa `clean_text()` trước, sau đó chạy lại audit và dùng báo cáo cuối cùng làm nguồn quyết định.

Bước 7 Tạo hai protocol dữ liệu riêng

Không nên chỉ có một phiên bản dataset.

Protocol A Standard processed

Dùng để báo cáo kết quả gần với official benchmark.

Thực hiện:

- Remap label.
- Sửa lỗi text.
- Giữ nguyên số dòng.
- Không xóa duplicate thông thường.
- Không dùng test để thay đổi train.

Tên file:

data/processed/standard_train.parquet
data/processed/standard_test.parquet

Protocol B Research clean

Dùng để:

- Tạo validation sạch.
- Phát triển LDTF.
- Làm ablation.
- So sánh các module.

Thực hiện:

- Remap label.

- Sửa lỗi text.
- Loại toàn bộ conflicting duplicate trong train.
- Chỉ giữ một bản trong mỗi nhóm same-label duplicate.
- Group near-duplicate trước khi chia validation.

Tên file:

data/processed/research_clean_pool.parquet

Khuyến nghị

Dùng Research clean để huấn luyện và phát triển model.

Cuối cùng báo cáo:

1. Kết quả official benchmark.
2. Kết quả trên protocol clean/decontaminated.

Phải ghi rõ protocol nào được sử dụng.

Bước 8 Phát hiện near-duplicate trong original train

Exact duplicate chưa đủ. Hai bài có thể gần như giống nhau nhưng khác vài từ.

Ví dụ:

Apple shares rise after earnings report.

Apple shares climb following its earnings report.

Mục tiêu

Không để một phiên bản nằm trong train và một phiên bản gần giống nằm trong validation.

Cách thực hiện

Không so sánh mọi cặp theo kiểu 120.000×120.000 , vì quá chậm.

Dùng một trong các cách:

MinHash + LSH
TF-IDF character n-gram + nearest neighbors
SimHash

Khuyến nghị:

Character n-gram: 3–5
Similarity threshold ban đầu: 0.90

Khi hai bài vượt ngưỡng, đưa chúng vào cùng một cluster.

Tạo:

near_duplicate_group_id

Audit thủ công

Kiểm tra:

- 100 cặp có similarity cao nhất.
- Các cặp gần ngưỡng 0.90.
- Các cluster chứa nhiều label khác nhau.

Không tự động loại toàn bộ near-duplicate. Mục tiêu chính là giữ cùng cluster trong cùng split.

File đầu ra

data/reports/near_duplicate_pairs.csv
data/manifests/near_duplicate_groups.parquet

Bước 9 Kiểm tra overlap giữa train và official test

Kiểm tra:

Exact hash overlap
Near-duplicate overlap
Label disagreement

Tuy nhiên, không dùng official test để chọn model.

Cách báo cáo nên dùng

Official test result

- Giữ nguyên 7.600 test samples.
- Dùng để so sánh với benchmark phổ biến.
- Báo cáo rõ có kiểm tra contamination.

Decontaminated test result

Tạo thêm một danh sách test không chứa:

- Exact overlap với train.
- Near-duplicate với train vượt ngưỡng đã khóa.

Tên:

data/manifests/decontaminated_test_ids.json

Kết quả này chỉ dùng làm phân tích phụ.

Không được làm

Không nhìn kết quả decontaminated test rồi quay lại chỉnh model.

Không chọn checkpoint bằng official test hoặc decontaminated test.

Bước 10 Chia training và validation sau khi cleaning

Thứ tự bắt buộc:

```
Raw train
↓
Clean text
↓
Remap labels
↓
Exact conflict removal
↓
Same-label deduplication
↓
```

Near-duplicate grouping

↓

Train-validation split

Không chia validation trước các bước trên.

Cách chia

Dùng group-aware stratification:

```
StratifiedGroupKFold(  
    n_splits=10,  
    shuffle=True,  
    random_state=42,  
)
```

Chọn một fold làm validation.

Kết quả gần:

90% train

10% validation

Label dùng để stratify:

label

Group dùng để khóa:

near_duplicate_group_id

Nếu một dòng không thuộc near-duplicate cluster, sử dụng text_hash của chính nó làm group riêng.

Kiểm tra sau split

Không có row_id trùng giữa train và validation

Không có text_hash trùng

Không có group_id xuất hiện ở cả hai tập

Tỷ lệ class gần bằng nhau

Tổng số dòng khớp

Chênh lệch tỷ lệ mỗi class giữa train và validation nên nhỏ hơn khoảng 1%.

Bước 11 Lưu split manifest cố định

Sau khi chia lần đầu, không được chia lại mỗi lần train.

Lưu:

```
data/manifests/train_ids.json
data/manifests/validation_ids.json
data/manifests/test_ids.json
data/manifests/split_metadata.json
```

split_metadata.json cần có:

```
{
  "seed": 42,
  "validation_ratio": 0.1,
  "split_method": "StratifiedGroupKFold",
  "dataset_protocol": "research_clean",
  "grouping_method": "near_duplicate_cluster",
  "similarity_threshold": 0.9,
  "label_mapping": {
    "0": "World",
    "1": "Sports",
    "2": "Business",
    "3": "Sci/Tech"
  }
}
```

Mọi model phải dùng cùng manifest:

```
BERT baseline frozen
BERT baseline fine-tuned
LDTF frozen
LDTF fine-tuned
Mọi ablation
Mọi seed
```

Bước 12 Phân tích chiều dài token

Dùng đúng tokenizer:

bert-base-uncased

Tokenize title và description dưới dạng một cặp:

```
tokenizer(  
    title_clean,  
    description_clean,  
    padding=False,  
    truncation=False,  
)
```

Thống kê:

mean
median
p90
p95
p99
maximum
percentage > 64
percentage > 96
percentage > 128
percentage > 192

Quyết định

Bắt đầu với:

MAX_LENGTH = 128

Giữ 128 nếu tỷ lệ vượt giới hạn không quá 2%.

Nếu vượt quá 2–5%, so sánh thêm 192.

Không tự động dùng 512.

File đầu ra

data/reports/token_length_report.json
data/reports/token_length_distribution.csv

Bước 13 Tokenize title và description như text pair

Không nên ghép:

title + " " + description

Thay vào đó dùng:

```
tokenizer(  
    title_clean,  
    description_clean,  
    truncation="only_second",  
    max_length=128,  
    padding=False,  
)
```

BERT sẽ tạo dạng:

[CLS] Title [SEP] Description [SEP]

Tại sao dùng only_second?

Title thường ngắn và chứa thông tin chủ đề quan trọng. Khi tổng độ dài vượt 128, chỉ nên cắt bớt Description thay vì cắt Title.

Batch cần có

```
input_ids  
attention_mask  
token_type_ids  
labels
```

token_type_ids giúp BERT phân biệt Title và Description.

Model baseline và multi-layer backbone phải hỗ trợ trường này.

bước 14 Sử dụng dynamic padding

Trong bước tokenize:

padding=False

Khi tạo batch, sử dụng:

```
DataCollatorWithPadding(  
    tokenizer=tokenizer,  
    padding=True,  
    pad_to_multiple_of=8  
)
```

pad_to_multiple_of=8 có thể giúp GPU Tensor Cores hoạt động hiệu quả hơn. Có thể tắt nếu nó làm tăng padding quá nhiều.

Không pad toàn bộ dataset lên 128 trước.

Ví dụ:

Batch 1: [16, 48]
Batch 2: [16, 80]
Batch 3: [16, 128]

Bước 15 Tạo DataLoader cho GPU Cloud

Training

shuffle=True
drop_last=False

Validation và test

shuffle=False
drop_last=False

Cấu hình ban đầu:

num_workers = 4
pin_memory = True
persistent_workers = True
prefetch_factor = 2

Nếu máy cloud ít CPU, giảm num_workers xuống 2.

Tạo generator có seed:

```
generator = torch.Generator()
generator.manual_seed(42)
```

Seed từng worker để DataLoader tái lập được.

Có thể dùng length bucketing để giảm lượng padding, nhưng chỉ thêm sau khi pipeline cơ bản chạy đúng.

Bước 16 Chạy batch sanity checks

Lấy một batch từ:

```
train_loader
validation_loader
test_loader
```

Kiểm tra:

```
assert input_ids.ndim == 2
assert attention_mask.ndim == 2
assert token_type_ids.ndim == 2
assert labels.ndim == 1
```

```
assert input_ids.shape == attention_mask.shape
assert input_ids.shape == token_type_ids.shape
assert input_ids.shape[0] == labels.shape[0]
```

```
assert labels.min() >= 0
assert labels.max() <= 3
```

```
assert input_ids.shape[1] <= 128
```

Attention mask chỉ được có:

```
0
1
```

Mỗi sample phải có ít nhất một valid token.

Decode ngẫu nhiên một vài sample để kiểm tra Title và Description vẫn hợp lý.

Bước 17 — Tạo bộ báo cáo cuối cùng

Trước khi train, phải có:

```
data/reports/
├── schema_report.json
├── cleaning_report.json
├── cleaning_samples.csv
├── class_distribution.csv
├── exact_duplicates.csv
├── conflicting_duplicates.csv
├── near_duplicate_pairs.csv
├── split_overlap_report.json
├── token_length_report.json
└── final_data_report.json
```

Processed data:

```
data/processed/
├── standard_train.parquet
├── standard_test.parquet
├── research_train.parquet
├── research_validation.parquet
└── research_test.parquet
```

Manifests:

```
data/manifests/
├── raw_checksums.txt
├── train_ids.json
├── validation_ids.json
├── test_ids.json
├── decontaminated_test_ids.json
├── removed_rows.csv
└── split_metadata.json
```

Bước 18 Chỉ bắt đầu training khi dataset PASS

Dataset chỉ PASS khi:

Schema errors = 0

Missing title = 0

Missing description = 0

Labels ngoài 0–3 = 0

Numeric HTML entity chưa sửa = 0

Backslash artifact chưa xử lý = 0

Conflicting exact duplicates trong research train = 0

Exact duplicate dư trong research train = 0

Train–validation text hash overlap = 0

Train–validation group overlap = 0

Tokenizer đúng bert-base-uncased

Tỷ lệ truncation tại 128 $\leq$ 2%

Attention mask hợp lệ

Split manifest đã lưu

Mọi model dùng cùng manifest

Nếu một điều kiện bắt buộc chưa đạt, không nên chạy full training.

Bước 19 Thứ tự huấn luyện sau khi data đã sạch

Không bắt đầu ngay bằng LDTF-BERT.

Thứ tự đúng:

1. Data pipeline smoke test
2. Tiny overfit test bằng BERT baseline
3. BERT baseline frozen

4. BERT baseline fine-tuned
5. Kiểm tra metric và confusion matrix
6. Multi-layer backbone smoke test
7. Label Query Bank
8. Token Router
9. Depth Router
10. Full LDTF-BERT
11. LDTF frozen
12. LDTF fine-tuned
13. Ablation
14. Chạy nhiều seed
15. Official test
16. Decontaminated test analysis

Trong quá trình phát triển, chỉ dùng train và validation.

Official test chỉ được chạy sau khi kiến trúc và hyperparameter đã được chốt.

Việc cụ thể bạn cần làm ngay bây giờ

Việc 1

Đưa hai file lên GPU Cloud:

data/raw/train.csv
data/raw/test.csv

Việc 2

Tạo một script duy nhất:

scripts/prepare_agnews.py

Không cần tạo quá nhiều script nhỏ.

Bên trong script có các function:

```
load_raw_csv()
validate_schema()
remap_labels()
clean_text()
create_text_hash()
audit_exact_duplicates()
build_near_duplicate_groups()
create_group_stratified_split()
analyze_token_lengths()
save_processed_data()
save_manifests()
save_reports()
```

Việc 3

Chạy audit nhưng chưa xóa dữ liệu:

```
python scripts/prepare_agnews.py \
  --train data/raw/train.csv \
  --test data/raw/test.csv \
  --mode audit-only
```

Đọc:

```
conflicting_duplicates.csv
cleaning_samples.csv
near_duplicate_pairs.csv
```

Việc 4

Khi audit hợp lệ, chạy chế độ build:

```
python scripts/prepare_agnews.py \
  --train data/raw/train.csv \
  --test data/raw/test.csv \
  --mode build \
  --validation-ratio 0.10 \
  --seed 42 \
```



```
--max-length 128 \  
--near-duplicate-threshold 0.90
```

Việc 5

Chạy smoke test DataLoader.

Việc 6

Chỉ khi mọi công PASS, bắt đầu huấn luyện BERT baseline.

Tóm tắt ngắn nhất

Thứ tự bạn phải làm là:

Giữ nguyên raw files
↓
Kiểm tra schema
↓
Chuyển label 1–4 thành 0–3
↓
Sửa HTML entity và backslash
↓
Tạo text hash
↓
Xử lý duplicate/conflict
↓
Group near-duplicate
↓
Chia train–validation theo group
↓
Luu manifest
↓
Tokenize Title + Description dạng pair
↓
MAX_LENGTH = 128
↓
Dynamic padding
↓
Batch sanity checks
↓
Train BERT baseline
↓
Train LDTF-BERT

Điểm quan trọng nhất là:

Không chia validation trước khi làm sạch, deduplicate và grouping. Không dùng official test để chọn model. Không đưa trực tiếp CSV thô vào tokenizer.